Software 3.0 and the Future of Programming — Andrej Karpathy

AI Conference · 2025 · 30 min

We've entered Software 3.0, where LLMs are programmable computers and prompting replaces coding, fundamentally changing what it means to build software and requiring us to rethink everything from development workflows to hiring practices.

The December Shift: When AI Started Actually Working

- December 2024 marked a stark transition where agentic tools stopped needing corrections and 'vibe coding' became real — chunks of code came out fine consistently for the first time
- This wasn't incremental improvement but a fundamental capability shift that many missed if they only experienced AI through ChatGPT-style interfaces
- The feeling of being 'behind as a programmer' came from realizing the paradigm had changed: side projects exploded as trust in agent output became justified

I can't remember the last time I corrected it and then I just trusted the system more and more and then I was vibe coding.

Software 3.0: Programming by Prompting

- Software 1.0 was explicit rules (writing code), Software 2.0 was learned weights (arranging datasets and training neural nets), Software 3.0 is prompting LLMs as programmable computers
- OpenClaw's installation isn't a bash script but text you copy-paste to an agent — the agent uses its intelligence to adapt to your environment, debug in the loop, and handle edge cases that would balloon traditional scripts
- Menu generation example: built a full Vercel app with OCR and image generation, then realized the Software 3.0 version is just 'take photo, give to Gemini, use Nanobanana to overlay' — the entire app was spurious, working in the old paradigm
- This isn't just faster programming — it's automating general information processing that couldn't exist before, like LLM-generated knowledge bases that recompile documents into new structures

Your neural network is doing more and more of the work and your prompt or context is just the image and the output is an image and there's no need to have any of the app in between.

The Verifiability Thesis: Why AI Is Jagged

- LLMs easily automate what you can verify because frontier labs train them in giant RL environments with verification rewards — they peak in math, code, and adjacent verifiable domains
- Jaggedness comes from what labs put in the data distribution and focus on: chess improved dramatically from GPT-3.5 to GPT-4 because someone added chess data, not from general capability growth
- State-of-the-art models can refactor 100k line codebases or find zero-days but tell you to walk to a car wash 50 meters away — this reveals you're at the mercy of what circuits were part of the RL
- If you're outside the data distribution, you need fine-tuning; if you're in a verifiable domain the labs haven't focused on, you can pull the RL lever yourself with diverse datasets

How is it possible that Opus 4.7 will simultaneously refactor a 100,000 line codebase and yet tells me to walk to this car wash? This is insane.

Vibe Coding vs Agentic Engineering

- Vibe coding raises the floor — everyone can build anything in software now, which is incredible for accessibility and prototyping
- Agentic engineering preserves the professional quality bar: no vulnerabilities, same responsibility, but going faster by coordinating spiky, stochastic, powerful agents correctly
- The ceiling on agentic capability is very high — 10x engineer is no longer the upper bound; people good at this peak far beyond 10x speedups
- Hiring must change: stop giving puzzles, give big projects like 'build a Twitter clone for agents, make it secure, then I'll use 10 Codex instances to try breaking it' — watch people build at scale with tooling

What Humans Still Own

- Agents are intern-level entities right now — humans must own aesthetics, judgment, taste, and oversight of the spec and top-level design
- Example failure: agent tried matching Stripe and Google accounts by email address instead of persistent user IDs — weird architectural choices that humans catch
- You no longer need to remember API details (keep_dims vs keep_dim, reshape vs permute) but must understand fundamentals like tensor views vs storage to avoid inefficiency
- Code quality is often bloaty with awkward abstractions — models struggle with simplification (micro-GPT project felt like pulling teeth, outside RL circuits)
- Understanding remains the bottleneck: you can outsource thinking but not understanding; you must still direct agents, know what to build and why

You can outsource your thinking but you can't outsource your understanding.

Animals vs Ghosts: What LLMs Actually Are

- LLMs aren't animal intelligences with intrinsic motivation, curiosity, or empowerment from evolution — they're 'ghosts' shaped by data and reward functions
- Yelling at them doesn't help; they're statistical simulation circuits with pre-training substrate plus RL bolted on top that increases jaggedness
- This framing helps build better mental models for competent use: be suspicious, understand what's likely to work, know how to modify behavior
- Not prescriptive with five obvious outcomes, but a mindset shift for working with fundamentally alien intelligence

The Agent-Native Future

- Everything is still written for humans — docs, infrastructure, deployment flows — but should be agent-first: 'What do I copy-paste to my agent?' not 'Go to this URL'
- Pet peeve: being told what to do instead of given agent-ready instructions; deployment friction (Vercel DNS config, service menus) should disappear
- Future test: give prompt 'build menu gen' and it deploys to internet without touching anything — decompose workloads into sensors and actuators over the world with agent-legible data structures
- Moving toward agent representation for people and organizations: 'my agent will talk to your agent' to handle meeting logistics and coordination details

I don't want to do anything. What is the thing I should copy paste to my agent?

What's Still Worth Learning

- Deep understanding remains essential even as intelligence gets cheap — you become the bottleneck for knowing what to build, why it matters, how to direct agents
- LLM knowledge bases and different projections onto information enhance understanding; synthetic data generation over fixed data provides insight
- Understanding is what lets you be a good director; LLMs don't excel at understanding, so humans remain uniquely in charge of that layer
- Tools that enhance understanding are incredibly interesting and exciting — this is the constraint that still matters